

Note: Write your answers at the designated place in the separate answer sheet provided. All the answers of this part (i.e. Part B) are based on the following case:

ABC Ltd. manufactures teddy-bears (*henceforth mentioned as TB*) of different varieties, where each TB is of same dimensions. Periodically, the company keeps on adding new such varieties. To showcase its TB's to the customers, company maintains a small show-room, where only one sample of each variety of TB is showcased. The show-room is so small that often it cannot accommodate the sample of all the varieties.

The company also maintains the popularity index of each variety of TB. For it, a list of all the TB's currently showcased is maintained. The position of TB in the list denotes the popularity index of corresponding TB; i.e. popularity of first TB in the list > popularity of second TB in the list > popularity of third TB in the list, and so on. This popularity index gets assigned/changes as per following cases:

Case 1: As soon as customer places an order for a TB (say A): The position of this TB in the list is changed so that it becomes the first one in the list.

Case 2: Company launches a new variety: Following two cases occur here:

Case 2a: If there is a space in the showroom, then this TB is showcased and it takes the first place in the list.

Case 2b: If there is no space in the showroom, then the TB with least popularity index is deleted from the list; it is removed from the showroom; the newly launched TB takes the first position in the list; and it is showcased in the showroom.

Assuming that list of TB's in the showroom is maintained as a linked list, following definitions maintain the information about the showcased TB's:

#define showcase_size 10 //Number of TB's which can be accommodated in the showcase. typedef enum {PINK=5,BROWN,WHITE} color; // Three colors of TB's	typedef struct { int bear_id; float bear_cost; color bear_color; } teddy_bear; //Information about a TB	typedef struct node { teddy_bear tb; struct node *next; } NODE; typedef struct node *Link; // Node of a Linked List	typedef struct { NODE *first; int size; } LIST; // Represent a Linked List
---	---	---	--

Q1. A file stores the record of all the TB's which company has manufactured till date. Each record contains three fields (i.e. bear_id, bear_cost, and bear_color), with one record per line. The first entry of the file indicates the total number of records (an integer) present in the file. Complete the following function which reads these records and stores them in an array of teddy_bear structure. Two arguments are passed to it: name of file, and a pointer **nor**. Initially, this pointer points to some garbage value. Within the function, the number of records in a file gets stored at the memory location pointed by **nor**. The function returns the base address of an array of teddy_bear structure.

```
teddy_bear* initialize (char *name[], int *nor)
{
    FILE *ptr;
    Q1(a). _____
    if (ptr == NULL) {printf ("Cannot open file");return 0;}
    Q1(b). Read the first entry of the file in variable nor
    teddy_bear *arr = (teddy_bear*) calloc(Q1(c). Complete the statement
    int i=0, temp = *nor;
    while (temp > 0)
    {
        Q1(d). Read a record about TB from the file
        temp--;
        i++;
    }
    Q1(e). _____
}
```

Q1 (f). Write the main function to call initialize(). The file name containing records of teddy bears is passed as command line argument to main function. The corresponding .c file containing main() and initialize() is compiled to produce the output file a.out. Finally, this output file is executed as follows:

./a.out record.dat

Q2. Assuming that showroom is not empty, this function, named NOVNIS, prints the colors of TB's not showcased in the showroom and returns number of such TB's. The input arguments passed to it are array of teddy_bear structures (populated in Q1 above), its size (nor) and the linked list of TB's in the showroom. [1+3+1+3 = 8M]

Q3. Complete the following recursive function, named printList. Using recursion, this function prints the bear_id of each TB in the list in reverse order, i.e. bear id of last node, followed by bear id of second last node, and so on. Input argument **head** is the pointer to the first node of the list.

```
Q3. void printList(Link head)
{
    [4M]
}
```

Q4. Complete the following function, **update_pop_index**, which takes a Linked List and a pointer to TB (whose priority index is to be updated) as input arguments. This function updates the popularity index as explained in Case-1 and Case-2 above. It returns the updated Linked List. [NOTE: This function uses another function, named **create_node()** at appropriate place(s)].

```
int NOVNIS (LIST tb, teddy_bear arr[], int nor)
{
    int i,count=0; //count stores number of TB's not in
                    showroom

    Link curr;
    for(i=0;i<nor;i++)
    {
        Q2(a).
        while(curr != NULL)
        {
            if(Q2(b). Write proper condition )
                break;
            else
                curr = curr->next;
        }
        if(Q2(c). Write condition if TB is not in showroom )
        {
            Q2(d). Using switch case statement, print the
                    color of TB not in the showroom.
            count++;
        }
    }
    return count;
}
```

```
LIST update_pop_index (LIST tb, teddy_bear *new)
{
    Link curr,prev;
    if( Q4 (a). Check the condition )
    {
        Link newBear = Q4 (b). Complete the statement ;
        tb.first = newBear;
        tb.size++;
        return tb;
    }
    curr = prev = tb.first;
    //Case-1
    while(curr != NULL)
    {
        if( Q4 (c). Check if TB is in showroom )
        {
            if( Q4 (d). Check the condition )
                return tb;
            else
            {
                Q4 (e).
                curr->next = tb.first;
                tb.first = curr;
                return tb;
            }
        }
        else
        {
            Q4 (f). Goto check next TB
        }
    }
} // END WHILE
```

```
// Case-2 starts here
Link newBear = Q4 (g). Complete the statement
curr = prev = tb.first;
if(tb.size == showcase_size)
{
    while(curr->next != NULL)
    {
        Q4 (h). Update pointer (s)
    }
    // Case 2b
    Q4 (i). Write required statement(s)
    free(curr);
    Q4 (j). Write required statement(s)
    return tb;
}
else if(tb.size < showcase_size)
{
    newBear->next = tb.first;
    tb.first = newBear;
    tb.size++;
    return tb;
}
} // END of FUNCTION
Link create_node (teddy_bear new)
{
    Link newBear;
    Q4 (k). Write required statement(s)
    return (newBear);
}
```

[1+2+2+1+1+2+2+2+1+1+2 = 17M]